

Refactoring Documentation

We fixed up this project by checking the code together and discussing what could be improved. Each person went through the code and noted down things that needed changing. Here's a list of what we found:

- 1) Refactor the ``d_gameContext`` variable in ``AdvanceOrder`` to retrieve from the ``GameContext`` singleton instead.
- 2) The ``GameContext`` class is becoming large. Consider creating new files to reduce the number of class variables in ``GameContext``. For example, create a ``MapContext`` class to store all map-related data, and have ``GameContext`` contain a ``MapContext`` as a data member.
- 3) In the startup service, ``loadmap``, move the implementation to a new file.
- 4) ``HelpView.java`` is hardcoded with applicable commands a user can enter. Consider generating this dynamically based on the current game phase.
- 5) Keep the logic for the ``isGameEnded`` function simple.
- 6) Combine several service classes that are similar. For example, merge ``ContinentService``, ``CountryService``, and ``MapService`` into a single class.
- 7) Implement adapters to read conquest map files.
- 8) Refactor the command router to not use enums and instead use inheritance/polymorphism to call the correct controller.
- 9) Rename ``conventOrder`` to ``convertOrder`` in ``Player.java``.
- 10) Change ``I_protentialWinner`` to ``I_potentialWinner`` in ``GameEngine.java``.
- 11) Standardize the style of enum fields for consistency. For example, update ``GamePhase.java`` to use a consistent style.
- 12) Remove or update several enums with unused values.
- 13) Create a service class to handle creating various orders. Currently, there are multiple methods to create orders in the ``Player`` class, which isn't very relevant to that class.
- 14) Rename ``conventDeployOrder`` to ``convertDeployOrder`` in ``Player.java``. We could also consider removing this method since it's not used. Maybe mark it as ``@Deprecated``?
- 15) Refactor the command router to not use enums and instead use inheritance/polymorphism to call the correct controller.

We streamlined the extensive list to focus on the most critical targets for implementation. This decision was made collectively during a team meeting where we agreed on the priorities. Here are five of the selected targets that we proceeded to implement:

- 1) Refactor the `d_gameContext` variable in `AdvanceOrder` to retrieve from the `GameContext` singleton instead.

Tests:

✓ AdvanceOrderTest (warzone.model)	117 ms
✓ testWithDiplomacyInCurrentTurn	117 ms

Reasoning:

It made sense to change this instance because the `GameContext` uses the singleton pattern on multiple occasions. This ensures that the game context instance is used consistently all over.

Before-After:

<pre>} // check if army number above zero if(d_numberOfArmies <= 0){ GenericView.printWarning("Could not perform the advance order with return false; } //Check if fromCountry is owned by the current player if(d_fromCountry.getOwner() == null !d_fromCountry.getOwner().equal GenericView.printWarning("Could not perform the advance order movi d_fromCountry.getCountryName() + ", because " + d_player.g return false; }</pre>	228 229 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242	<pre>} // check if army number above zero if(d_numberOfArmies <= 0){ GenericView.printWarning("Could not perform the advance order wit return false; } //Check if fromCountry is owned by the current player if(d_fromCountry.getOwner() == null !d_fromCountry.getOwner().equal GenericView.printWarning("Could not perform the advance order mov d_fromCountry.getCountryName() + ", because " + d_player. return false; }</pre>	219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242
<pre>//check if DIPLOMACY if(d_toCountry.getOwner() != null && this.d_player != null && this.d_gameContext.isDiplomacyInCurrentTurn(d_player, d_toC GenericView.printWarning(String.format("The player [%s] and [%s] a return false; }</pre>	243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266	<pre>//check if DIPLOMACY if(d_toCountry.getOwner() != null && this.d_player != null && this.getGameContext().isDiplomacyInCurrentTurn(d_player, d GenericView.printWarning(String.format("The player [%s] and [%s] a return false; }</pre>	267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290

- 2) `GameEngine.play()` was renamed to `GameEngine.playSingleMode()`

Tests:

✓ GameEngineServiceTest (warzone.service)	188 ms
✓ testTournamentModeEmptyMapList	98 ms
✓ testInvalidGameCanStartNotEnoughPlayers	44 ms
✓ testTournamentModeValid	14 ms
✓ testInvalidGameCanStartNoCountriesAssigned	14 ms
✓ testTournamentModeEmptyPlayerStrategiesList	4 ms
✓ testValidGameCanStart	14 ms

Reasoning:

We would want to draw attention to the method's renaming here, however the logic was somewhat altered. `Play()` got unclear because we had to implement tournament features for this build. Now, `playSingleMode()` makes it obvious what the method does.

Before:

```
/**
 * If the game turn is greater than 100, the game will end.
 *
 * @return true if the game can end.
 */
public boolean play() {

    if(! isReadyToStart())
        return false;
    if(this.d_gameContext.getIsDemoMode())
        startTurn();
    else {
        while(!isGameEnded())
            startTurn();
    }
    return true;
}
```

After:

```
/**
 * Play single mode
 * @return if it is finished
 */
1 usage  ± Khushi2111 +1
public boolean playSingleMode() {
    GenericView.println("Single Mode is Starting");
    int l_turnCounter = 0;

    while(l_turnCounter < WarzoneProperties.getWarzoneProperties().getMaxTurnNumberPerGame() && !isGameEnded()) {
        startTurn();
        l_turnCounter++;
    }
    renderAndUpdateGameResult();

    GenericView.println("Single Mode is Ended after executing [" + l_turnCounter + "] Turn. ");

    return true;
}
```

3) Keep the logic for the `isGameEnded` function simple.

Test:

```
/**
 * Test if a player wins the game by conquering all the countries
 */
// Vrushabh Savani
@Test
public void testIsGameEnded() {

    Continent l_continent = new Continent( p_continentID: 1, p_continentName: "Continent-1");

    //Create 2 players
    Player p1 = new Player( p_name: "p1");
    Player p2 = new Player( p_name: "p2");
    p1.setIsAlive(true);
    p2.setIsAlive(true);
    d_gameContext.getPlayers().put("p1", p1);
    d_gameContext.getPlayers().put("p2", p2);

    //Create 2 countries
    Country country1 = new Country( p_countryID: 1, p_countryName: "country1");
    Country country2 = new Country( p_countryID: 2, p_countryName: "country2");
    country1.setContinent(l_continent);
    country2.setContinent(l_continent);
    country1.addNeighbor(country2);
    country2.addNeighbor(country1);
    country1.setArmyNumber(3);
    country2.setArmyNumber(0);
    d_gameContext.getCountries().put(1, country1);
    d_gameContext.getCountries().put(2, country2);

    //Assign one country to each player
    p1.getConqueredCountries().put(1, country1);
    p2.getConqueredCountries().put(2, country2);
    country1.setOwner(p1);
    country2.setOwner(p2);

    //Create an advance order -> p1's country1 attacks p2's country2
    p1.getOrders().add(p1.createAdvanceOrder(new String[] {"advance", "country1", "country2", "3"}));

    d_gameEngine.setPhase(new OrderExecution(d_gameEngine));

    //Assert that the game has not yet ended (the order has not executed yet)
    assertTrue( condition: d_gameEngine.isGameEnded() == false);
}
```

Reasoning:

We refactored the `isGameEnded()` code to make it run more quickly and with better code quality because it was complicated.

Before:

```
/**
 * This method will determine if the game whether can end.
 * @return true if the current state satisfy the end condition:
 * 1. there is just one player left 2. the number of game turn is greater than 100.
 */
public boolean isGameEnded() { 9 usages 1 Vrushabh Savani +3
    if(this.d_gamePhase.getGamePhase() == GamePhase.MAPEEDITOR)
        return false;

    //check and update PlayerStatus
    //set p_isloser = true, when the player does not have any country
    int l_alivePlayers = 0;
    Player l_potentialWinner = null;
    for(Player l_player :d_gameContext.getPlayers().values() ){
        if(l_player.getConqueredCountries().size() > 0) {
            l_player.setIsAlive(true);
            l_potentialWinner = l_player;
            l_alivePlayers ++;
        }
    }
    if(l_alivePlayers <= 1){
        GenericView.println("----- Game End");
        if(l_alivePlayers == 1)
            GenericView.printSuccess(p_text: "player " + l_potentialWinner.getName() + " wins the game.");
        else
            GenericView.printSuccess(p_text: "All the player died.");
        GenericView.println("----- Reboot the game");
        this.reboot();
        return true;
    }
    else
        return false;
}
```

After:

```
/**
 * This method will determine if the game whether can end.
 * @return true if the current state satisfy the end condition:
 * 1. there is just one player left 2. the number of game turn is greater than 100.
 */
14 usages 1 Pritesh
public boolean isGameEnded() {
    return isGameEnded(p_isShowResult: false);
}

/**
 * This method will determine if the game whether can end.
 * update player 's status
 * @param p_isShowResult is show result
 * @return true if the current state satisfy the end condition:
 * 1. there is just one player left 2. the number of game turn is greater than 100.
 */
2 usages 1 Pritesh +1
public boolean isGameEnded(boolean p_isShowResult) {
    if(this.d_gamePhase.getGamePhase() == GamePhase.MAPEEDITOR)
        return false;

    //check and update PlayerStatus
    //set p_isloser = true, when the player does not have any country
    int l_alivePlayers = 0;
    Player l_potentialWinner = null;
    for(Player l_player :d_gameContext.getPlayers().values() ){
        if(l_player.getConqueredCountries().size() > 0) {
            l_player.setIsAlive(true);
            l_potentialWinner = l_player;
            l_alivePlayers ++;
        }
        else {
            l_player.setIsAlive( false );
        }
    }
    if(l_alivePlayers <= 1){
        return true;
    }
    else
        return false;
}
```

4) Added adapter to read conquest map files.

Tests:

```
/**
 * test map file "simple-starwar"
 */
// Vrushabh Savani
@Test
public void testLoadConquestMap() {
    assertTrue(d_conquestMapReader.loadConquestMap( p_fileName: "simple-starwar.map"));
    assertEquals(d_gameContext.getContinents().size(), actual: 2);

    assertEquals(d_gameContext.getContinents().get(1).getCountries().size(), actual: 2);
    assertEquals(d_gameContext.getContinents().get(2).getCountries().size(), actual: 3);

    assertEquals(d_gameContext.getCountries().size(), actual: 5);
    assertEquals(d_gameContext.getCountries().get(1).getNeighbors().size(), actual: 1);
    assertEquals(d_gameContext.getCountries().get(2).getNeighbors().size(), actual: 3);
    assertEquals(d_gameContext.getCountries().get(3).getNeighbors().size(), actual: 2);
    assertEquals(d_gameContext.getCountries().get(4).getNeighbors().size(), actual: 1);
    assertEquals(d_gameContext.getCountries().get(5).getNeighbors().size(), actual: 1);

    assertEquals(d_gameContext.getCountries().get(1).getContinent().getContinentID(), actual: 1);
    assertEquals(d_gameContext.getCountries().get(2).getContinent().getContinentID(), actual: 1);
    assertEquals(d_gameContext.getCountries().get(3).getContinent().getContinentID(), actual: 2);
    assertEquals(d_gameContext.getCountries().get(4).getContinent().getContinentID(), actual: 2);
    assertEquals(d_gameContext.getCountries().get(5).getContinent().getContinentID(), actual: 2);
}

/**
 * test invalid map file "no-such-conquest-map"
 */
// Vrushabh Savani
@Test
public void testLoadConquestMap2() { assertFalse(d_conquestMapReader.loadConquestMap( p_fileName: "no-such-conquest-map.map")); }
```

Reasoning:

This was a high priority target because it was specified in the build specifications. Users can now play the game using both the Domination and Conquest map files thanks to this refactoring.

Before:

```
14 public class MapServiceAdapter extends MapService{ no usages 1 Vrushabh Savani
18     ConquestMapWriter d_conquestMapWriter; 2 usages
19     /**
20      * conquest map reader
21      */
22     ConquestMapReader d_conquestMapReader; 2 usages
23
24     /**
25      * the constructor of the class
26      * @param p_gameContext the current game context
27      */
28     public MapServiceAdapter(GameContext p_gameContext, ConquestMapWriter p_conquestMapWriter, ConquestMapReader p_conquestMapReader)
29     {
30         super(p_gameContext);
31         this.d_conquestMapWriter = p_conquestMapWriter;
32         this.d_conquestMapReader = p_conquestMapReader;
33     }
34
35     /**
36      * This method will save maps with 'conquest' format
37      * @param p_fileName the name of the map
38      * @return true if successfully
39      */
40     public boolean saveMap(String p_fileName) throws IOException { 1 Vrushabh Savani
41         return d_conquestMapWriter.saveConquestMap(p_fileName);
42     }
43
44     /**
45      * This method will perform editmap to maps with 'conquest' format
46      * @param p_fileName the name of the map
47      * @return true if successfully
48      */
49     public boolean editMap(String p_fileName){ return d_conquestMapReader.loadConquestMap(p_fileName); }
51 }
```

After:

```
/**
 * This method will determine the map type and instance the d_StartupService with according
 * objects by map file.
 */
no usages new *
private void determineMapTypeFromGameContext() {
    GameContext l_gameContext = GameContext.getGameContext();
    if (l_gameContext.getMapType() == MapType.CONQUEST) {
        d_mapService = new MapServiceAdapter(l_gameContext, new ConquestMapWriter(l_gameContext), new ConquestMapReader(l_gameContext));
    }
    else if (l_gameContext.getMapType() == MapType.DOMINATION) {
        d_mapService = new MapService(l_gameContext);
    }
}
```

5) New StartupService constructor

Test:

✓ StartupServiceTest (warzone.service)	952 ms
✓ testAssignReinforcementsNoOwnedContinentsFourteenOwnedC	425 ms
✓ checkGamePhaseAfterSaveAndLoadGameContext	433 ms
✓ testLoadMap	32 ms
✓ testAssignReinforcementsNoOwnedContinentsNoOwnedCountrie	29 ms
✓ willNotLoadNonExistGameContext	3 ms
✓ testAssignReinforcementsOneOwnedContinentFiveOwnedCountri	26 ms
✓ testAssignCountries	4 ms

Reasoning:

When coding, this extra constructor offers greater versatility.

Before:

```
public class StartupService { 16 usages  LoveBorad15 +3

    /**
     * game context
     */
    private GameContext d_gameContext; 27 usages

    /**
     * log entry buffer
     */
    private LogEntryBuffer d_logEntryBuffer; 7 usages

    /**
     * This constructor can initiate the game context of current instance.
     * @param p_gameContext the current game context
     */
    public StartupService(GameContext p_gameContext) { 10 usages  Vrushabh Savani +1
        d_gameContext = p_gameContext;
        d_logEntryBuffer = d_gameContext.getLogEntryBuffer();
    }
}
```


After:

```
/**
 * This class will provide controllers with service associating with startup
 * @author Pritesh
 */
22 usages  📄 harsh-hirapara +3
public class StartupService implements Serializable {

    /**
     * game context
     */
    30 usages
    private GameContext d_gameContext;

    /**
     * game engine
     */
    3 usages
    private GameEngine d_gameEngine;

    /**
     * log entry buffer
     */
    3 usages
    private LogEntryBuffer d_logEntryBuffer;

    /**
     * Dominate Map Handler
     */
    4 usages
    private DominateMapHandler d_mapHandler;

    /**
     * This constructor can initiate the game context of current instance.
     * @param p_gameContext the current game context
     */
    12 usages  📄 LoveBorad15 +2
    public StartupService(GameContext p_gameContext) {
        d_gameEngine = GameEngine.getGameEngine(p_gameContext);
        d_gameContext = p_gameContext;
        d_logEntryBuffer = d_gameContext.getLogEntryBuffer();
        d_mapHandler = new DominateMapHandler(d_gameContext);
    }

    /**
     * This constructor can initiate the game context of current instance.
     * @param p_gameEngine the current game engine
     */
    2 usages  📄 LoveBorad15 +1
    public StartupService(GameEngine p_gameEngine) {
        d_gameEngine = p_gameEngine;
        d_gameContext = p_gameEngine.getGameContext();
        d_logEntryBuffer = d_gameContext.getLogEntryBuffer();
        d_mapHandler = new DominateMapHandler(d_gameContext);
    }
}
```